DESIGN CODE OF 8-BIT ALU (VHDL)

src/alu_8bit_vhdl.vhd [VHDL]

```
----------------------------------------------------------------------------------
-- Company:
-- Engineer:
--
-- Create Date: 09/17/2026 04:43:19 PM
-- Design Name:
-- Module Name: alu_8bit_vhdl - Behavioral
-- Project Name:
-- Target Devices:
-- Tool Versions:
-- Description:
--
-- Dependencies:
--
-- Revision:
-- Revision 0.01 - File Created
-- Additional Comments:
--
----------------------------------------------------------------------------------


library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity alu_8bit_vhdl is
    Port ( op : in STD_LOGIC_VECTOR (3 downto 0);
           a : in signed (7 downto 0);
           b : in signed (7 downto 0);
           c : out signed (15 downto 0);
           v : out STD_LOGIC);
end alu_8bit_vhdl;

architecture Behavioral of alu_8bit_vhdl is
begin

    process(a , b , op)

```

```
47          variable ae : signed(15 downto 0);
48          variable be : signed(15 downto 0);
49
50          variable limit_temp : std_logic;
51
52
53      begin
54
55          ae := TO_SIGNED(TO_INTEGER(a) , 16);
56          be := TO_SIGNED(TO_INTEGER(b) , 16);
57
58
59
60          case op is
61
62          when x"0" =>
63              c <= ae + be;
64              v <= '0';
65
66          when x"1" =>
67              c <= ae - be;
68              v <= '0';
69
70          when x"2" =>
71              c <= a * b;
72              v <= '0';
73
74          when x"3" =>
75              if ( b = x"00") then
76                  c <= x"ffff";
77                  v <= '1';
78              else
79                  c <= ae/be;
80                  v <= '0';
81
82              end if;
83
84          when x"4" =>
85              if ( b = x"00") then
86                  c <= x"ffff";
87                  v <= '1';
88              else
89                  c <= ae mod be;
90                  v <= '0';
91
92              end if;
93
94          when x"5" =>
95              if ( b = x"00") then
96                  c <= x"ffff";
97                  v <= '1';
```

```
98              else
99                  c <= ae rem be;
100                 v <= '0';
101 
102             end if;
103 
104         when x"6" =>
105             c <= x"00" & (not a );
106             v <= '0';
107 
108         when x"7" =>
109             c <= x"00" & (a and b);
110             v <= '0';
111 
112         when x"8" =>
113             c <= x"00" & (a xor b) ;
114             v <= '0';
115 
116         when x"9" =>
117             c <= (ae and x"00ff") srl TO_INTEGER(be);
118             v <= '0';
119 
120         when x"a" =>
121             c <= (ae and x"00ff") sll TO_INTEGER(be);
122             v <= '0';
123 
124         when x"b" =>
125             c <= x"00" & (a ror 1);
126             v <= '0';
127 
128         when x"c" =>
129             c <= x"00" & (a rol 1);
130             v <= '0';
131 
132         when x"d" =>
133             if a(7) = '1' then
134                 if a  < -32 then
135                     c <= x"ffff";
136                     v <= '1';
137                 else
138                     c <= TO_SIGNED(TO_INTEGER(ae*ae*ae) , 16);
139                     v <= '0';
140                 end if;
141             else
142                 if a  >= 32 then
143                     c <= x"ffff";
144                     v <= '1';
145                 else
146                     c <= TO_SIGNED(TO_INTEGER(ae*ae*ae) , 16);
147                     v <= '0';
148                 end if;
```

```
            end if;

        when x"e" =>
            c <= TO_SIGNED(TO_INTEGER(ae*ae) , 16) - (b/2);
            v <= '0';

        when x"f" =>
            c <= TO_SIGNED(TO_INTEGER(be*be) , 16) - (b rem 17);
            v <= '0';

        when others =>
            c <= TO_SIGNED(0 , 16);
            v <= '0';

        end case;
    end process;

end Behavioral;
```

TESTBENCH CODE OF 8-BIT ALU (VHDL)

src/alu8_vhdl_tb.vhd [VHDL]

```
----------------------------------------------------------------------------------
-- Company:
-- Engineer:
--
-- Create Date: 09/17/2026 04:43:19 PM
-- Design Name:
-- Module Name: alu8_vhdl_tb - Behavioral
-- Project Name:
-- Target Devices:
-- Tool Versions:
-- Description:
--
-- Dependencies:
--
-- Revision:
-- Revision 0.01 - File Created
-- Additional Comments:
--
----------------------------------------------------------------------------------


library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity alu8_vhdl_tb is
end alu8_vhdl_tb;
```

```
28
29  architecture Behavioral of alu8_vhdl_tb is
30      signal op : STD_LOGIC_VECTOR(3 downto 0) := (others => '0');
31      signal a  : signed(7 downto 0) := (others => '0');
32      signal b  : signed(7 downto 0) := (others => '0');
33      signal c  : signed(15 downto 0);
34      signal v  : STD_LOGIC;
35  begin
36
37      alu_vhdl : entity work.alu_8bit_vhdl
38          port map (
39              op => op,
40              a  => a,
41              b  => b,
42              c  => c,
43              v  => v
44          );
45
46      stimulus : process
47      begin
48          op <= x"0"; a <= to_signed(25, 8);  b <= to_signed(15, 8);  wait for 10
    ns;
49          op <= x"0"; a <= to_signed(-50, 8); b <= to_signed(20, 8);  wait for 10
    ns;
50          op <= x"0"; a <= to_signed(127, 8); b <= to_signed(1, 8);   wait for 10
    ns;
51
52          op <= x"1"; a <= to_signed(50, 8);  b <= to_signed(20, 8);  wait for 10
    ns;
53          op <= x"1"; a <= to_signed(-20, 8); b <= to_signed(30, 8);  wait for 10
    ns;
54
55          op <= x"2"; a <= to_signed(12, 8);  b <= to_signed(10, 8);  wait for 10
    ns;
56          op <= x"2"; a <= to_signed(-12, 8); b <= to_signed(10, 8);  wait for 10
    ns;
57          op <= x"2"; a <= to_signed(-128, 8); b <= to_signed(-128, 8); wait for 1
    0 ns;
58
59          op <= x"3"; a <= to_signed(25, 8);  b <= to_signed(5, 8);   wait for 10
    ns;
60          op <= x"3"; a <= to_signed(-25, 8); b <= to_signed(4, 8);   wait for 10
    ns;
61          op <= x"3"; a <= to_signed(25, 8);  b <= to_signed(0, 8);   wait for 10
    ns;
62
63          op <= x"4"; a <= to_signed(17, 8);  b <= to_signed(5, 8);   wait for 10
    ns;
64          op <= x"4"; a <= to_signed(-17, 8); b <= to_signed(5, 8);   wait for 10
    ns;
65          op <= x"4"; a <= to_signed(17, 8);  b <= to_signed(0, 8);   wait for 10
    ns;
66
67          op <= x"5"; a <= to_signed(17, 8);  b <= to_signed(5, 8);   wait for 10
    ns;
```

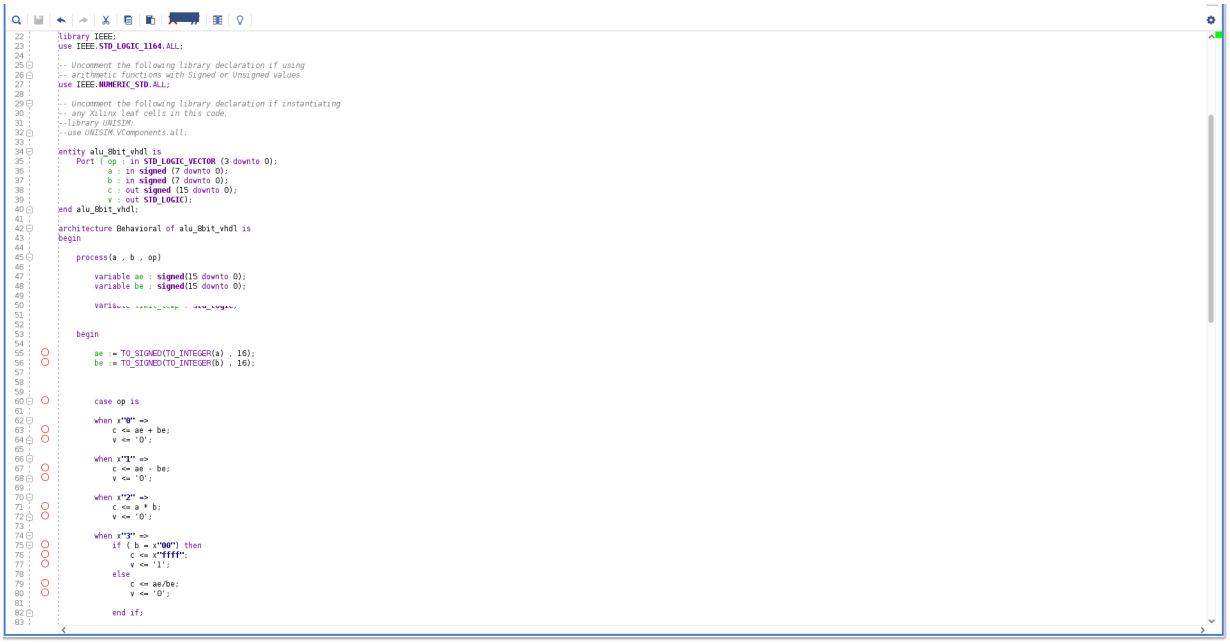
```
68          op <= x"5"; a <= to_signed(-17, 8); b <= to_signed(5, 8);   wait for 10
ns;
69          op <= x"5"; a <= to_signed(17, 8);  b <= to_signed(0, 8);   wait for 10
ns;
70
71          op <= x"6"; a <= to_signed(0, 8);   b <= to_signed(0, 8);   wait for 10
ns;
72          op <= x"6"; a <= to_signed(-1, 8);  b <= to_signed(0, 8);   wait for 10
ns;
73
74          op <= x"7"; a <= to_signed(85, 8);  b <= to_signed(15, 8);  wait for 10
ns;
75          op <= x"7"; a <= to_signed(-1, 8);  b <= to_signed(85, 8);  wait for 10
ns;
76
77          op <= x"8"; a <= to_signed(85, 8);  b <= to_signed(15, 8);  wait for 10
ns;
78          op <= x"8"; a <= to_signed(-1, 8);  b <= to_signed(85, 8);  wait for 10
ns;
79
80          op <= x"9"; a <= to_signed(129, 8); b <= to_signed(1, 8);   wait for 10
ns;
81          op <= x"9"; a <= to_signed(255, 8); b <= to_signed(8, 8);   wait for 10
ns;
82
83          op <= x"A"; a <= to_signed(1, 8);   b <= to_signed(7, 8);   wait for 10
ns;
84          op <= x"A"; a <= to_signed(1, 8);   b <= to_signed(15, 8);  wait for 10
ns;
85
86          op <= x"B"; a <= to_signed(129, 8); b <= to_signed(0, 8);   wait for 10
ns;
87          op <= x"B"; a <= to_signed(1, 8);   b <= to_signed(0, 8);   wait for 10
ns;
88
89          op <= x"C"; a <= to_signed(129, 8); b <= to_signed(0, 8);   wait for 10
ns;
90          op <= x"C"; a <= to_signed(64, 8);  b <= to_signed(0, 8);   wait for 10
ns;
91
92          op <= x"D"; a <= to_signed(3, 8);   b <= to_signed(0, 8);   wait for 10
ns;
93          op <= x"D"; a <= to_signed(-3, 8);  b <= to_signed(0, 8);   wait for 10
ns;
94          op <= x"D"; a <= to_signed(32, 8);  b <= to_signed(0, 8);   wait for 10
ns;
95          op <= x"D"; a <= to_signed(-33, 8); b <= to_signed(0, 8);   wait for 10
ns;
96
97          op <= x"E"; a <= to_signed(12, 8);  b <= to_signed(5, 8);   wait for 10
ns;
98          op <= x"E"; a <= to_signed(-12, 8); b <= to_signed(-5, 8);  wait for 10
ns;
99
100         op <= x"F"; a <= to_signed(3, 8);   b <= to_signed(5, 8);   wait for 10
ns;
```

```
        op <= x"F"; a <= to_signed(-3, 8);  b <= to_signed(5, 8);   wait for 10 ns;

        wait;
    end process;

end Behavioral;
```

SCREENSHOT OF VHDL CODE AND TESTBENCH



```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Uncomment the following library declaration if using
-- arithmetic functions with Signed or Unsigned values
use IEEE.NUMERIC_STD.ALL;

-- Uncomment the following library declaration if instantiating
-- any Xilinx leaf cells in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity alu_8bit_vhdl is
    Port ( op : in STD_LOGIC_VECTOR (3 downto 0);
           a : in signed (7 downto 0);
           b : in signed (7 downto 0);
           c : out signed (15 downto 0);
           v : out STD_LOGIC);
end alu_8bit_vhdl;

architecture Behavioral of alu_8bit_vhdl is
begin

    process(a , b , op)

        variable ae : signed(15 downto 0);
        variable be : signed(15 downto 0);

        variable temp : STD_LOGIC;

    begin

        ae := TO_SIGNED(TO_INTEGER(a) , 16);
        be := TO_SIGNED(TO_INTEGER(b) , 16);


        case op is

        when x"0" =>
            c <= ae + be;
            v <= '0';

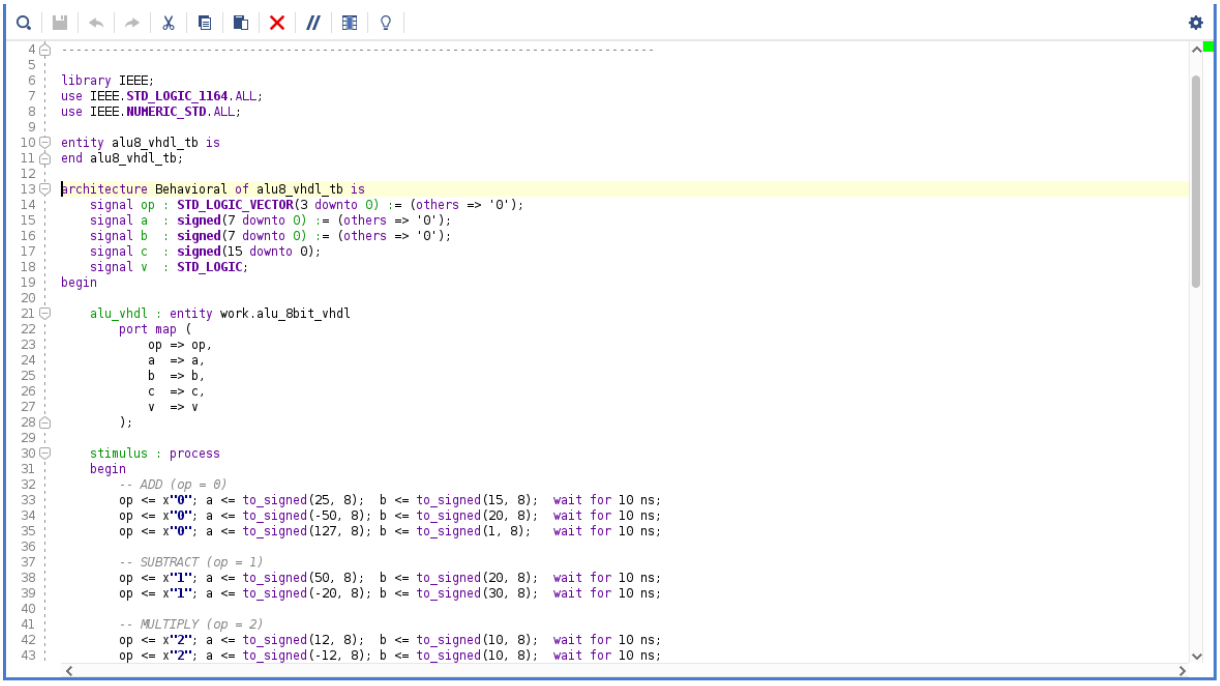
        when x"1" =>
            c <= ae - be;
            v <= '0';

        when x"2" =>
            c <= a * b;
            v <= '0';

        when x"3" =>
            if ( b = x"00") then
                c <= x"ffff";
                v <= '1';
            else
                c <= ae/be;
                v <= '0';

            end if;
```

Figure 1: ALU VHDL Design Code



```
----------------------------------------------------------------------------------

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity alu8_vhdl_tb is
end alu8_vhdl_tb;

architecture Behavioral of alu8_vhdl_tb is
    signal op : STD_LOGIC_VECTOR(3 downto 0) := (others => '0');
    signal a  : signed(7 downto 0) := (others => '0');
    signal b  : signed(7 downto 0) := (others => '0');
    signal c  : signed(15 downto 0);
    signal v  : STD_LOGIC;
begin

    alu_vhdl : entity work.alu_8bit_vhdl
        port map (
            op => op,
            a  => a,
            b  => b,
            c  => c,
            v  => v
        );

    stimulus : process
    begin
        -- ADD (op = 0)
        op <= x"0"; a <= to_signed(25, 8);  b <= to_signed(15, 8);  wait for 10 ns;
        op <= x"0"; a <= to_signed(-50, 8); b <= to_signed(20, 8);  wait for 10 ns;
        op <= x"0"; a <= to_signed(127, 8); b <= to_signed(1, 8);   wait for 10 ns;

        -- SUBTRACT (op = 1)
        op <= x"1"; a <= to_signed(50, 8);  b <= to_signed(20, 8);  wait for 10 ns;
        op <= x"1"; a <= to_signed(-20, 8); b <= to_signed(30, 8);  wait for 10 ns;

        -- MULTIPLY (op = 2)
        op <= x"2"; a <= to_signed(12, 8);  b <= to_signed(10, 8);  wait for 10 ns;
        op <= x"2"; a <= to_signed(-12, 8); b <= to_signed(10, 8);  wait for 10 ns;
```

Figure 2: ALU VHDL Testbench Code

VHDL SIMULATION WAVEFORM

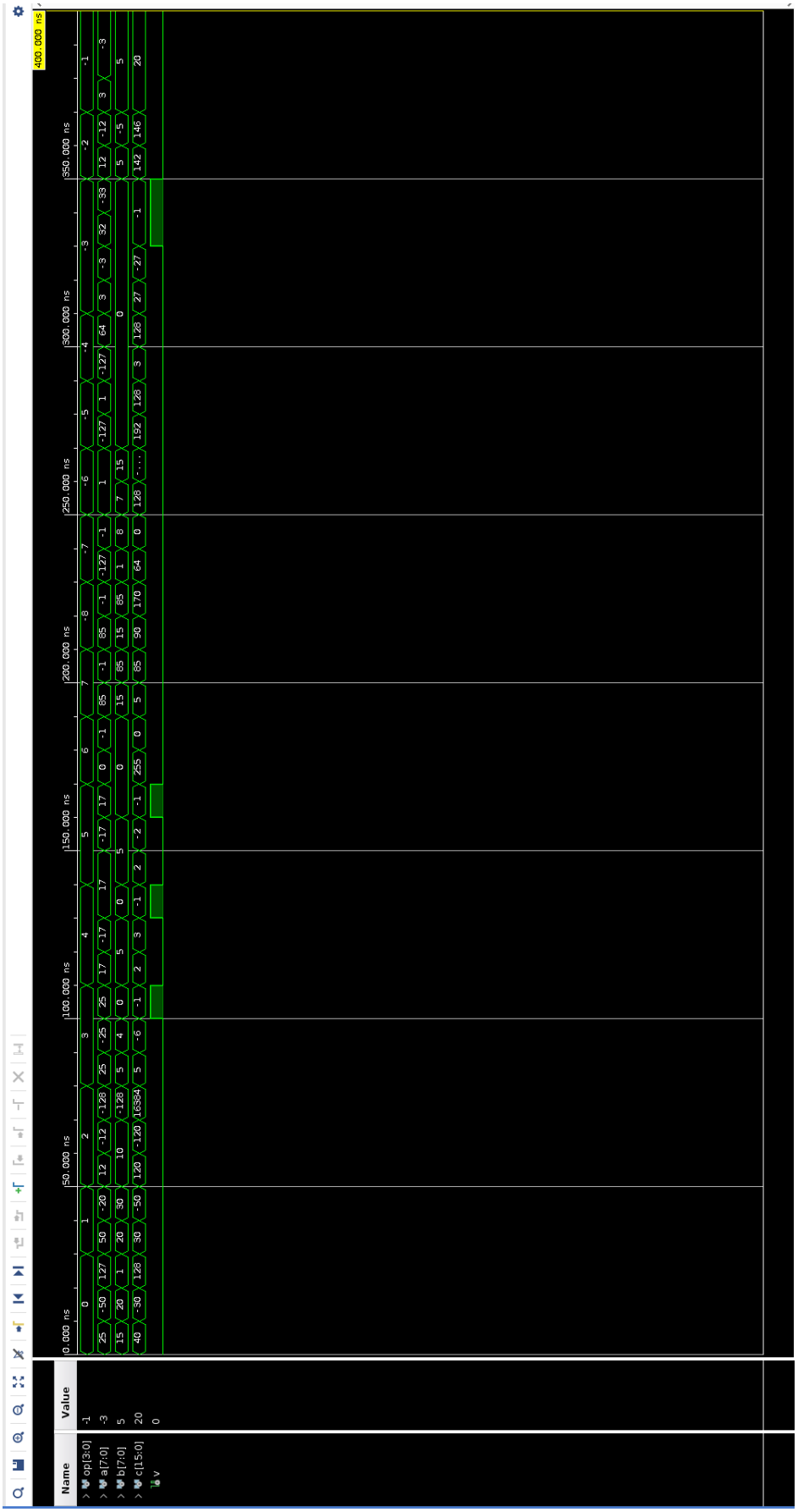


Figure 3: ALU VHDL Simulation Waveform

DESIGN CODE OF 8-BIT ALU (VERILOG)

src/alu_8bit_verilog.v [Verilog]

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 09/17/2026 04:43:19 PM
// Design Name:
// Module Name: alu_8bit_verilog
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////


module alu_8bit_verilog(
    input [3:0] op,
    input [7:0] a,
    input [7:0] b,
    output reg [15:0] c,
    output reg v
    );

    always @(*) begin
        v <= 1'b0;
    case(op)

        4'h0 : begin
              c <=  a + b;
        end
        4'h1 : c <=  a - b;
        4'h2 : c <= a*b;
        4'h3 : begin
              if ( b == 0) begin
                  c <= 16'hffff;
                  v <= 1'b1;
                  end
              else begin
                  c <= a/b;
```

```
                    v <= 1'b0;
                end
        end
        4'h4 : begin
                if ( b == 0) begin
                    c <= 16'hffff;
                    v <= 1'b1;
                    end
                else begin
                    c <= ((a%b) + b)%b;
                    v <= 1'b0;
                end
        end
        4'h5 : begin
                if ( b == 0) begin
                    c <= 16'hffff;
                    v <= 1'b1;
                    end
                else begin
                    c <= a%b;
                    v <= 1'b0;
                end
        end
        4'h6 : c <= ~a;
        4'h7 : c <= a & b;
        4'h8 : c <= a ^ b;
        4'h9 : c <= a >> b;
        4'ha : c <= a << b;
        4'hb : c <= (a >> 1) | (a << 7);
        4'hc : c <= (a << 1) | (a >> 7);
        4'hd : begin
                if ( ($signed(a) < -8'sd32) || ( $signed(a) >= 8'sd32)) begin
                    c <= 16'hffff;
                    v <= 1'b1;
                end
                else begin
                    c <= a*a*a;
                    v <= 1'b0;
                end
        end
        4'he : c <= a*a - b/2;
        4'hf : c <= b*b - b%17;
        default : c <= 16'h0000;
    endcase


    end


endmodule
```

TESTBENCH CODE OF 8-BIT ALU (VERILOG)

src/alu8_verilog_tb.v [Verilog]

```verilog
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 09/17/2026 04:43:19 PM
// Design Name:
// Module Name: alu8_verilog_tb
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////


module alu8_verilog_tb;

    reg  [3:0] op;
    reg  [7:0] a;
    reg  [7:0] b;
    wire [15:0] c;
    wire v;

    alu_8bit_verilog uut (
        .op(op),
        .a(a),
        .b(b),
        .c(c),
        .v(v)
    );

    initial begin
        op = 4'h0;
        a  = 8'h00;
        b  = 8'h00;
        #10;

        op = 4'h0; a = 8'd25;  b = 8'd15;  #10;
        op = 4'h0; a = 8'hCE;  b = 8'd20;  #10;
```

```
        op = 4'h0; a = 8'h7F;  b = 8'd1;   #10;
        op = 4'h0; a = 8'hFF;  b = 8'd1;   #10;

        op = 4'h1; a = 8'd50;  b = 8'd20;  #10;
        op = 4'h1; a = 8'd20;  b = 8'd50;  #10;
        op = 4'h1; a = 8'h80;  b = 8'h7F;  #10;

        op = 4'h2; a = 8'd12;  b = 8'd10;  #10;
        op = 4'h2; a = 8'hF4;  b = 8'd10;  #10;
        op = 4'h2; a = 8'h80;  b = 8'h80;  #10;
        op = 4'h2; a = 8'h7F;  b = 8'h7F;  #10;

        op = 4'h3; a = 8'd25;  b = 8'd5;   #10;
        op = 4'h3; a = 8'hE7;  b = 8'd4;   #10;
        op = 4'h3; a = 8'd25;  b = 8'd0;   #10;
        op = 4'h3; a = 8'd0;   b = 8'd7;   #10;

        op = 4'h4; a = 8'd17;  b = 8'd5;   #10;
        op = 4'h4; a = 8'hEF;  b = 8'd5;   #10;
        op = 4'h4; a = 8'd17;  b = 8'd0;   #10;

        op = 4'h5; a = 8'd17;  b = 8'd5;   #10;
        op = 4'h5; a = 8'hEF;  b = 8'd5;   #10;
        op = 4'h5; a = 8'd17;  b = 8'd0;   #10;

        op = 4'h6; a = 8'h00; b = 8'h00; #10;
        op = 4'h6; a = 8'hFF; b = 8'h00; #10;
        op = 4'h6; a = 8'h80; b = 8'h00; #10;

        op = 4'h7; a = 8'h55; b = 8'h0F; #10;
        op = 4'h7; a = 8'hFF; b = 8'h55; #10;
        op = 4'h7; a = 8'h00; b = 8'hFF; #10;

        op = 4'h8; a = 8'h55; b = 8'h0F; #10;
        op = 4'h8; a = 8'hFF; b = 8'h55; #10;
        op = 4'h8; a = 8'hAA; b = 8'hAA; #10;

        op = 4'h9; a = 8'h81; b = 8'd0;  #10;
        op = 4'h9; a = 8'h81; b = 8'd1;  #10;
        op = 4'h9; a = 8'h81; b = 8'd7;  #10;
        op = 4'h9; a = 8'hFF; b = 8'd8;  #10;

        op = 4'hA; a = 8'h81; b = 8'd0;  #10;
        op = 4'hA; a = 8'h01; b = 8'd7;  #10;
        op = 4'hA; a = 8'h01; b = 8'd8;  #10;

        op = 4'hB; a = 8'h81; b = 8'd0;  #10;
        op = 4'hB; a = 8'h01; b = 8'd0;  #10;
        op = 4'hB; a = 8'hFF; b = 8'd0;  #10;

        op = 4'hC; a = 8'h81; b = 8'd0;  #10;
```

```
        op = 4'hC; a = 8'h40; b = 8'd0;  #10;
        op = 4'hC; a = 8'hFF; b = 8'd0;  #10;

        op = 4'hD; a = 8'd0;   b = 8'd0; #10;
        op = 4'hD; a = 8'd3;   b = 8'd0; #10;
        op = 4'hD; a = 8'hFD;  b = 8'd0; #10;
        op = 4'hD; a = 8'd31;  b = 8'd0; #10;
        op = 4'hD; a = 8'd32;  b = 8'd0; #10;
        op = 4'hD; a = 8'hE0;  b = 8'd0; #10;
        op = 4'hD; a = 8'hDF;  b = 8'd0; #10;

        op = 4'hE; a = 8'd12;  b = 8'd5;   #10;
        op = 4'hE; a = 8'hF4;  b = 8'hFB;  #10;
        op = 4'hE; a = 8'h80;  b = 8'h7F;  #10;

        op = 4'hF; a = 8'd3;   b = 8'd5;   #10;
        op = 4'hF; a = 8'hFD;  b = 8'd5;   #10;
        op = 4'hF; a = 8'd0;   b = 8'd5;   #10;

        #10;
        $finish;
    end

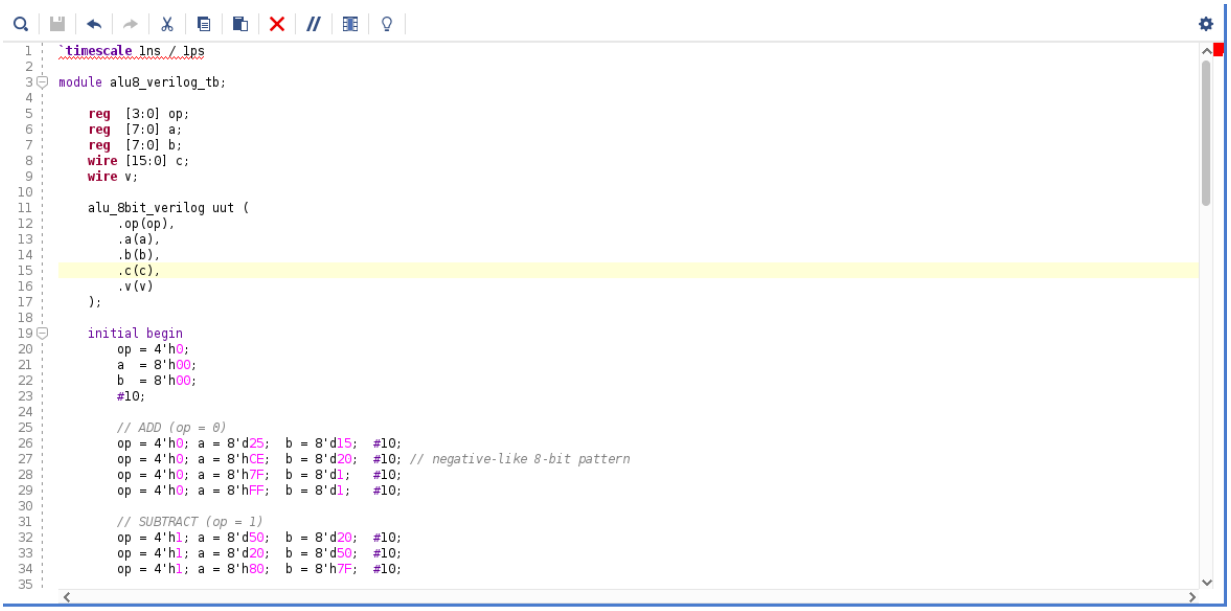
endmodule
```

SCREENSHOT OF VERILOG CODE AND TESTBENCH



```
21
22
23  module alu_8bit_verilog(
24      input [3:0] op,
25      input [7:0] a,
26      input [7:0] b,
27      output reg [15:0] c,
28      output reg v
29      );
30
31
32
33      always @(*) begin
34          v <= 1'b0;
35      case(op)
36
37          4'h0 : c <= {8'h00 , a + b};
38          4'h1 : c <= {8'h00 , a - b};
39          4'h2 : c <= a*b;
40          4'h3 : begin
41                 if ( b == 0) begin
42                     c <= 16'hffff;
43                     v <= 1'b1;
44                     end
45                 else begin
46                     c <= a/b;
47                     v <= 1'b0;
48                 end
49          end
50          4'h4 : begin
51                 if ( b == 0) begin
52                     c <= 16'hffff;
53                     v <= 1'b1;
54                     end
55                 else begin
56                     c <= ((a%b) + b)%b;
57                     v <= 1'b0;
58                 end
59          end
60          4'h5 : begin
```

Figure 4: ALU Verilog Design Code



```
1   `timescale 1ns / 1ps
2
3   module alu8_verilog_tb;
4
5       reg  [3:0] op;
6       reg  [7:0] a;
7       reg  [7:0] b;
8       wire [15:0] c;
9       wire v;
10
11      alu_8bit_verilog uut (
12          .op(op),
13          .a(a),
14          .b(b),
15          .c(c),
16          .v(v)
17      );
18
19      initial begin
20          op = 4'h0;
21          a  = 8'h00;
22          b  = 8'h00;
23          #10;
24
25          // ADD (op = 0)
26          op = 4'h0; a = 8'd25;  b = 8'd15;  #10;
27          op = 4'h0; a = 8'hCE;  b = 8'd20;  #10; // negative-like 8-bit pattern
28          op = 4'h0; a = 8'h7F;  b = 8'd1;   #10;
29          op = 4'h0; a = 8'hFF;  b = 8'd1;   #10;
30
31          // SUBTRACT (op = 1)
32          op = 4'h1; a = 8'd50;  b = 8'd20;  #10;
33          op = 4'h1; a = 8'd20;  b = 8'd50;  #10;
34          op = 4'h1; a = 8'h80;  b = 8'h7F;  #10;
35
```

Figure 5: ALU Verilog Testbench Code

VERILOG SIMULATION WAVEFORM

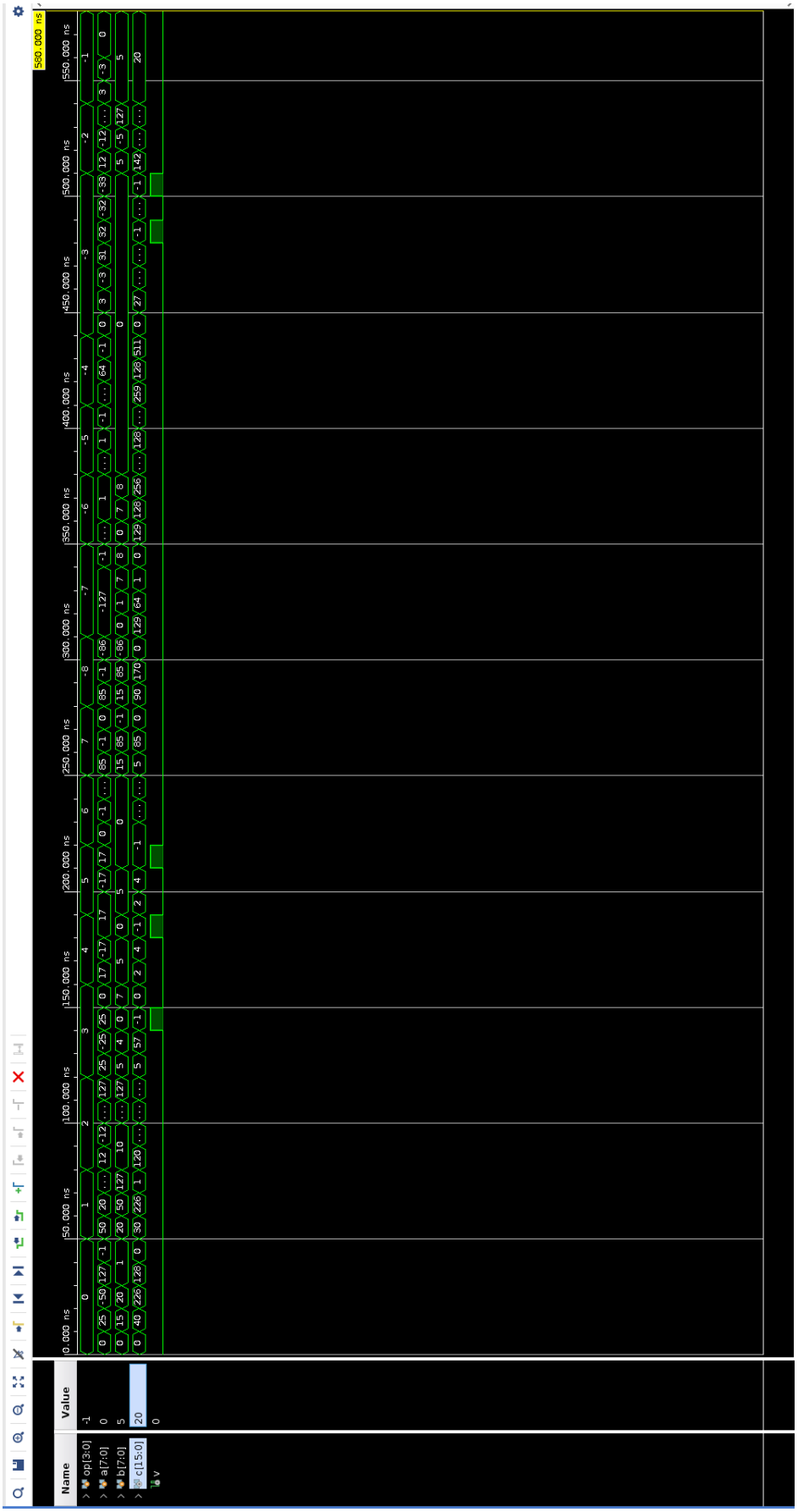


Figure 6: ALU Verilog Simulation Waveform